RV32I ISA reference sheet

HIPV core: base integer instruction set

1. Instruction formats

```
    31         25 24   20 19   15 14    12 11          7 6       0
 R |    funct7   |  rs2  |  rs1  | funct3 |      rd     |  opcode |
 I |      imm[11:0]      |  rs1  | funct3 |      rd     |  opcode |
 S |  imm[11:5]  |  rs2  |  rs1  | funct3 |   imm[4:0]  |  opcode |
 B | imm[12|10:5]|  rs2  |  rs1  | funct3 | imm[4:1|11] |  opcode |
 U |              imm[31:12]             |      rd     |  opcode |
 J |        imm[20|10:1|11|19:12]         |      rd     |  opcode |
```

All immediates sign-extend from instruction bit 31. B and J immediates have an implicit bit 0 = 0.

2. Opcodes

Opcode (bin)	Name	Instructions
0110011	OP	R-type ALU: add, sub, sll, slt, sltu, xor, srl, sra, or, and
0010011	OP-IMM	I-type ALU: addi, slti, sltiu, xori, ori, andi, slli, srli, srai
0000011	LOAD	lb, lh, lw, lbu, lhu
0100011	STORE	sb, sh, sw
1100011	BRANCH	beq, bne, blt, bge, bltu, bgeu
1101111	JAL	jal
1100111	JALR	jalr
0110111	LUI	lui
0010111	AUIPC	auipc
0001111	MISC-MEM	fence
1110011	SYSTEM	ecall, ebreak

3. Instruction table (37 instructions)

Integer register-register (R-type, opcode 0110011)

Instr	funct7	funct3	Operation
add rd, rs1, rs2	0000000	000	rd = rs1 + rs2
sub rd, rs1, rs2	0100000	000	rd = rs1 - rs2
sll rd, rs1, rs2	0000000	001	rd = rs1 << rs2[4:0]
slt rd, rs1, rs2	0000000	010	rd = (rs1 < rs2) signed
sltu rd, rs1, rs2	0000000	011	rd = (rs1 < rs2) unsigned
xor rd, rs1, rs2	0000000	100	rd = rs1 ^ rs2
srl rd, rs1, rs2	0000000	101	rd = rs1 >> rs2[4:0] (logical)
sra rd, rs1, rs2	0100000	101	rd = rs1 >> rs2[4:0] (arithmetic)
or rd, rs1, rs2	0000000	110	rd = rs1 OR rs2
and rd, rs1, rs2	0000000	111	rd = rs1 & rs2

Integer register-immediate (I-type, opcode 0010011)

Instr	imm[11:5]	funct3	Operation
addi rd, rs1, imm	imm[11:5]	000	rd = rs1 + sext(imm)
slti rd, rs1, imm	imm[11:5]	010	rd = (rs1 < sext(imm)) signed
sltiu rd, rs1, imm	imm[11:5]	011	rd = (rs1 < sext(imm)) unsigned
xori rd, rs1, imm	imm[11:5]	100	rd = rs1 ^ sext(imm)
ori rd, rs1, imm	imm[11:5]	110	rd = rs1 OR sext(imm)
andi rd, rs1, imm	imm[11:5]	111	rd = rs1 & sext(imm)
slli rd, rs1, shamt	0000000	001	rd = rs1 << shamt
srli rd, rs1, shamt	0000000	101	rd = rs1 >> shamt (logical)
srai rd, rs1, shamt	0100000	101	rd = rs1 >> shamt (arithmetic)

Loads (I-type, opcode 0000011) and stores (S-type, opcode 0100011)

Instr	funct3	Operation
lb rd, off(rs1)	000	rd = sext(mem8[rs1 + off])
lh rd, off(rs1)	001	rd = sext(mem16[rs1 + off])
lw rd, off(rs1)	010	rd = mem32[rs1 + off]
lbu rd, off(rs1)	100	rd = zext(mem8[rs1 + off])
lhu rd, off(rs1)	101	rd = zext(mem16[rs1 + off])
sb rs2, off(rs1)	000	mem8[rs1 + off] = rs2[7:0]
sh rs2, off(rs1)	001	mem16[rs1 + off] = rs2[15:0]
sw rs2, off(rs1)	010	mem32[rs1 + off] = rs2

Branches (B-type, opcode 1100011)

Instr	funct3	Condition (taken: PC = PC + sext(imm))
beq rs1, rs2, label	000	rs1 == rs2
bne rs1, rs2, label	001	rs1 != rs2
blt rs1, rs2, label	100	rs1 < rs2 signed
bge rs1, rs2, label	101	rs1 >= rs2 signed
bltu rs1, rs2, label	110	rs1 < rs2 unsigned
bgeu rs1, rs2, label	111	rs1 >= rs2 unsigned

Jumps, upper immediates, system

Instr	Opcode	Operation
jal rd, label	1101111	rd = PC + 4; PC = PC + sext(imm) (J-type)
jalr rd, off(rs1)	1100111	rd = PC + 4; PC = (rs1 + sext(off)) & ~1 (I-type, funct3 000)
lui rd, imm	0110111	rd = imm << 12 (U-type)
auipc rd, imm	0010111	rd = PC + (imm << 12) (U-type)
fence	0001111	memory ordering (funct3 000)
ecall	1110011	environment call (imm = 0, rest zero)
ebreak	1110011	breakpoint (imm = 1, rest zero)

4. Registers and ABI names

Caller-saved: ra, t0-t6, a0-a7. Callee-saved: sp, s0-s11. gp and tp are not saved.

Reg	ABI	Role	Reg	ABI	Role	Reg	ABI	Role	Reg	ABI	Role
x0	zero	zero	x8	s0/fp	saved/fp	x16	a6	arg	x24	s8	saved
x1	ra	ret addr	x9	s1	saved	x17	a7	arg	x25	s9	saved
x2	sp	stack	x10	a0	arg/ret	x18	s2	saved	x26	s10	saved
x3	gp	global	x11	a1	arg/ret	x19	s3	saved	x27	s11	saved
x4	tp	thread	x12	a2	arg	x20	s4	saved	x28	t3	temp
x5	t0	temp	x13	a3	arg	x21	s5	saved	x29	t4	temp
x6	t1	temp	x14	a4	arg	x22	s6	saved	x30	t5	temp
x7	t2	temp	x15	a5	arg	x23	s7	saved	x31	t6	temp

5. Common pseudo-instructions

Pseudo	Expands to
`nop`	`addi x0, x0, 0`
`li rd, imm`	`addi rd, x0, imm` (12-bit) or `lui` + `addi`
`mv rd, rs`	`addi rd, rs, 0`
`not rd, rs`	`xori rd, rs, -1`
`neg rd, rs`	`sub rd, x0, rs`
`seqz rd, rs`	`sltiu rd, rs, 1`
`snez rd, rs`	`sltu rd, x0, rs`
`sltz rd, rs` / `sgtz rd, rs`	`slt rd, rs, x0` / `slt rd, x0, rs`
`beqz rs, label` / `bnez rs, label`	`beq rs, x0, label` / `bne rs, x0, label`
`blez` / `bgez` / `bltz` / `bgtz rs, label`	`bge x0, rs` / `bge rs, x0` / `blt rs, x0` / `blt x0, rs`
`bgt` / `ble` / `bgtu` / `bleu rs, rt, label`	`blt rt, rs` / `bge rt, rs` / `bltu rt, rs` / `bgeu rt, rs`
`j label`	`jal x0, label`
`jal label`	`jal ra, label`
`jr rs`	`jalr x0, 0(rs)`
`jalr rs`	`jalr ra, 0(rs)`
`ret`	`jalr x0, 0(ra)`
`call label`	`auipc ra, hi` + `jalr ra, lo(ra)`
`la rd, symbol`	`auipc rd, hi` + `addi rd, rd, lo`
`lw rd, symbol`	`auipc rd, hi` + `lw rd, lo(rd)`

6. Handy facts

- 32 registers, 32 bits wide; `x0` always reads 0 and ignores writes.
- Shifts use only the low 5 bits of the shift amount (`rs2[4:0]` or `shamt`).
- There is no `subi`: use `addi` with a negative immediate.
- Loading a 32-bit constant: `lui rd, hi20` then `addi rd, rd, lo12`, where `hi20` must be incremented by one when bit 11 of the constant is set, because `addi` sign-extends.
- Branch range is +/-4 KiB (13-bit signed, even offsets); `jal` range is +/-1 MiB (21-bit signed, even offsets).
- Instructions are 32-bit and must be 4-byte aligned (no C extension).
- Misaligned load/store behavior is implementation defined.

7. HIPV core: decode status

Instructions	Status in the IDU
All R-type, I-ALU, loads, stores, branches, `jal`, `jalr`, `lui`, `auipc`	decoded (see `docs/IDU_decode_cheatsheet.md`)
`fence`, `ecall`, `ebreak`, CSR instructions	not implemented: decode to an all-zero control word and pass through as a nop (no illegal-instruction trap)